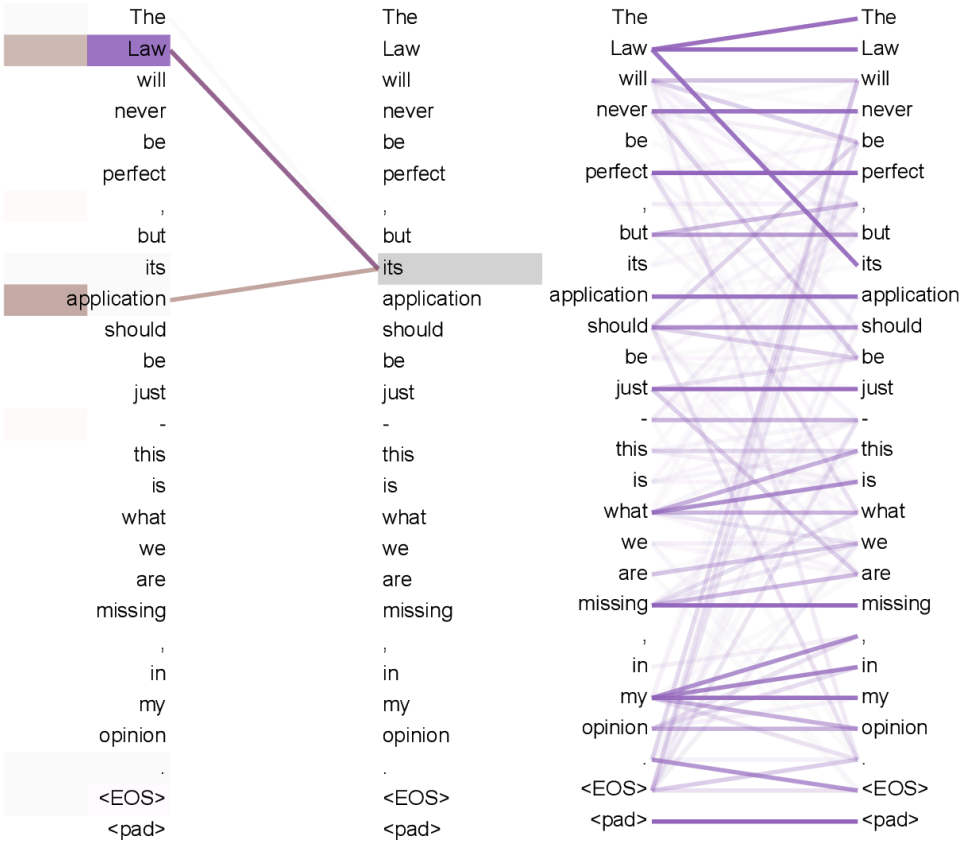
Mini-Transformer: um “ChatGPT simplificado” em RISC-V

Este documento descreve os projetos da disciplina de Introdução à Arquitetura de Computadores 2025/2026 (LEIC-A e LEIC-T).



Fonte: Vaswani et al., Attention Is All You Need*, NIPS 2017*

Histórico de versões

- v1.0: Enunciado publicado (20.04.2026).
- v1.1: Alterações ao Projeto 1

Introdução

Nos últimos anos, surgiram aplicações capazes de efetuar tarefas complexas relacionadas com texto – por exemplo, responder a perguntas, traduzir frases ou gerar texto automaticamente. Ferramentas como o ChatGPT[1] são exemplos desse tipo de sistemas. Muitos destes sistemas baseiam-se numa arquitetura de Inteligência Artificial baseada em redes neuronais[2] chamada **Transformer**[3].

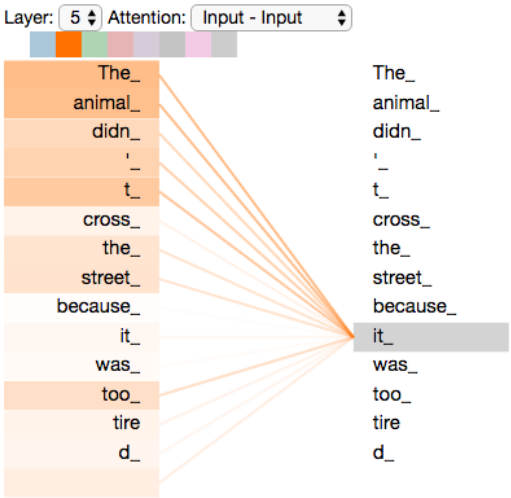
A ideia central destes modelos é o mecanismo de **atenção (attention)**: *para cada elemento da entrada (por exemplo, para cada palavra numa frase), o mecanismo decide quais são os outros elementos (palavras) mais relevantes*. Isto permite interpretar corretamente o significado das palavras tendo em conta o seu **contexto**, algo essencial em linguagem natural.

Vamos motivar este mecanismo de atenção com um exemplo[4]. Considera a seguinte frase:

"The animal didn't cross the street because it was too tired."

Uma questão importante é: a que se refere a palavra "it"? Será a "animal" ou a "street"? Para responder corretamente, é necessário perceber o contexto da frase. A palavra "tired" está mais associada a "animal" do que a "street", pelo que "it" se refere ao animal. Isto mostra que, ao interpretar uma palavra, precisamos de olhar para outras palavras da frase, e que nem todas são igualmente importantes.

Um modelo baseado em atenção atribui diferentes níveis de importância (ou **pesos**) às palavras. Neste exemplo, a palavra "animal" terá uma ligação mais forte com "it", como ilustrado na figura seguinte:



Para um computador fazer este tipo de raciocínio, cada palavra é representada por um **vetor de valores numéricos**, e os pesos de atenção são calculados através de operações matemáticas sobre esses vetores, em particular **produtos internos**. Em modelos reais, estes vetores são aprendidos automaticamente a

[1] https://chatgpt.com
[2] https://developers.google.com/machine-learning/crash-course/neural-networks
[3] *Vaswani et al.,* Attention Is All You Need, *NIPS 2017*
[4] https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/

partir de grandes quantidades de dados. Existem vários detalhes adicionais que tornam estes modelos muito eficazes, mas, para já, a ideia essencial é a seguinte: *o mecanismo de atenção permite perceber o contexto de uma palavra numa frase através do cálculo de pesos de relevância entre as palavras, usando operações relativamente simples sobre vetores*.

Neste projeto, iremos focar-nos numa tarefa específica destes modelos: **a previsão do próximo token (palavra)**. Dada uma sequência de palavras, o objetivo do modelo é prever qual será a palavra seguinte. Esta é a tarefa fundamental por trás de muitos sistemas de geração de texto, incluindo chatbots como o ChatGPT.

Mini-Transformer

Neste projeto, irão implementar um modelo de Inteligência Artificial muito simples, inspirado no mecanismo de atenção apresentado anteriormente, a que chamaremos **Mini-Transformer**. O objetivo não é reproduzir um Transformer completo, mas sim construir um pequeno sistema que aplica, na prática, a ideia central da atenção: *comparar elementos de uma frase entre si, calcular a sua relevância, e usar essa informação para produzir um resultado*.

1. **Entrada do modelo: frase → tokens → vetores**

O modelo recebe como entrada uma **frase**. Para poder ser processada por um computador, a frase é dividida em unidades chamadas **tokens**. Neste projeto, vamos assumir que cada token corresponde a uma **palavra**.

O modelo utiliza um vocabulário fixo, isto é, um conjunto limitado de palavras (tokens) que conhece. A cada palavra do vocabulário é associada a um índice inteiro.

Por exemplo, a frase "the animal didn't cross the street" pode ser representada como uma sequência de tokens:

["the", "animal", "didn't", "cross", "the", "street"]

Usando o vocabulário fornecido, cada token é convertido no índice correspondente, obtendo-se uma sequência de inteiros. Por exemplo:

[2, 3, 4, 5, 2, 6]

No entanto, estes inteiros servem apenas para identificar palavras e não contêm informação sobre o seu significado. Para que o modelo consiga comparar palavras e perceber relações entre elas, cada índice é convertido num vetor de números (designado por *embedding*). Por exemplo:

"the" → 2 → [0.1, 0.3, 0.7]

"animal" → 3 → [0.8, 0.2, 0.1]

…

Estes vetores permitem ao modelo calcular semelhanças entre palavras (por exemplo, através de produtos internos), o que é essencial para o funcionamento do mecanismo de atenção.

Assim, a frase passa a ser representada por uma **sequência de vetores**, que pode também ser vista como uma **matriz**. Chamando ***E*** a essa representação, temos:

$$\boldsymbol{E} = [[0.1, 0.3, 0.7], [0.8, 0.2, 0.1], \ldots]$$

Cada elemento (linha) de ***E*** corresponde ao vetor (*embedding*) de um token da frase.

Notem que os *embeddings* não são definidos ao acaso. Em modelos reais, são aprendidos automaticamente a partir de dados, ajustando os vetores de forma a que palavras que aparecem em contextos semelhantes tenham representações semelhantes. Neste projeto, no entanto, esses vetores serão fornecidos.

2. Como funciona o Mini-Transformer?

Recordemos que a sequência de vetores de entrada pode ser representada por uma **matriz *E***, onde:

- cada elemento (linha) de ***E*** é o vetor (*embedding*) de um token (palavra)
- se a frase tiver n tokens e embeddings de dimensão d, então ***E*** tem dimensão $n \times d$

Assim, ***E* representa a frase inteira**.

A partir desta matriz ***E***, o Mini-Transformer executa os seguintes passos:

Passo 1. Gerar representações intermédias (*Q*, *K*, *V*)

Para cada token i (isto é, para cada vetor i em ***E***), o modelo calcula três novos vetores:

- Q_i (Query)
- K_i (Key)
- V_i (Value)

Assim, ***Q***, ***K*** e ***V*** são matrizes com n linhas (uma por token), onde a linha i corresponde aos vetores Q_i, K_i e V_i, respetivamente.

Estas representações são obtidas através de transformações lineares simples:

```
Q = E × W_Q

K = E × W_K

V = E × W_V
```

onde:

- ***E*** é a matriz de vetores de entrada

- ***W_Q***, ***W_K***, ***W_V*** são matrizes de pesos (fornecidas no projeto)

Interpretação. As matrizes ***W_Q***, ***W_K*** e ***W_V*** transformam os embeddings originais em três representações diferentes. Em particular:

- As matrizes ***Q*** e ***K*** são usadas para calcular relevâncias entre palavras
- A matriz ***V*** representa a informação que será utilizada para construir a saída

Ou seja, cada palavra passa a ter três versões diferentes:

- uma para comparar com as outras (***Q***)
- uma para ser comparada (***K***)
- uma que contém a informação a usar na saída (***V***)

Passo 2. Calcular relevâncias.

Depois de calcular ***Q*** e ***K***, o modelo mede a relevância entre palavras. Para isso, cada vetor de ***Q*** é comparado com todos os vetores ***K***, usando o **produto interno**. Se quisermos analisar a palavra na posição ***i***, então calculamos:

```
score(i, j) = Q_i · K_j
```

para todos os valores de ***j***.

Isto significa que escolhemos uma palavra da frase, pegamos no seu vetor $\boldsymbol{Q_i}$, comparamos esse vetor com os vetores ***Kj*** de todas as palavras da frase e obtemos uma lista de valores, um por palavra.

Quanto maior for o valor obtido, maior é a relevância dessa palavra para a palavra que estamos a analisar.

Passo 3. Selecionar o mais relevante.

Depois de calcular todos os valores de relevância para uma palavra, o modelo seleciona o maior deles. Isto equivale a escolher a palavra da frase que é considerada mais relevante para a palavra que estamos a analisar. Neste projeto, essa escolha é feita usando uma função de máximo (**argmax**).

Passo 4. Produzir a saída.

Depois de escolher a palavra mais relevante, o modelo vai buscar o vetor $\boldsymbol{V_i}$ correspondente a essa palavra. Esse vetor $\boldsymbol{V_i}$ passa então a ser a nova representação da palavra que estamos a analisar.

Ou seja, ***Q*** e ***K*** servem para decidir *qual* a palavra mais relevante e ***V*** fornece a informação que será efetivamente usada.

Passo 5. Decisão final (classificação).

Neste projeto, para produzir a saída final, usamos o vetor correspondente à última palavra da frase. A partir desse vetor, o modelo escolhe uma palavra do vocabulário ***como previsão para o próximo token*** (isto é, a próxima palavra da frase). Essa escolha é feita comparando esse vetor com os vetores das palavras do vocabulário e selecionando a palavra mais semelhante.

Nota: Embora usemos apenas o vetor correspondente à última palavra para produzir a saída final, esse vetor já incorpora informação das restantes palavras da frase. Isto acontece porque, nos passos anteriores, cada palavra foi comparada com todas as outras através do mecanismo de atenção.

3. Exemplo numérico

Para ilustrar o funcionamento do modelo, vamos considerar a frase original, mas simplificada, considerando apenas os três primeiros tokens:

["the", "animal", "didn't"]

O objetivo é descobrir qual a palavra seguinte (tarefa de previsão do próximo token), que está na base de muitos sistemas de geração de texto.

Passo 1. Representação de entrada

Suponhamos que os embeddings são:

E = [

[1,0], ← "the"

[0,1], ← "animal"

[1,1] ← "didn't"

]

Considere as seguintes matrizes de pesos:

W_Q = [

[1,0],

[0,1]

]

W_K = [

[-2,0],

[1,1]

]

W_V = [

[0,1],

[2,0]

]

Passo 2. Cálculo de *Q*, *K* e *V*

Q = [

[1,0], ← "the"

[0,1], ← "animal"

[1,1] ← "didn't"

]

K = [

[-2,0], ← "the"

[1,1], ← "animal"

[-1,1] ← "didn't"

]

V = [

[0,1],

[2,0],

[2,1]

]

Passo 3. Calcular relevância (para a última palavra)

A última palavra é "didn't", logo:

Q_3 = [1,1]

Calculamos os produtos internos:

$Q_3 \cdot K_1 = [1,1]\cdot[-2,0] = -2 \leftarrow$ "the"

$Q_3 \cdot K_2 = [1,1]\cdot[1,1] = 2 \leftarrow$ "animal"

$Q_3 \cdot K_3 = [1,1]\cdot[-1,1] = 0 \leftarrow$ "didn't"

Passo 4. Selecionar o mais relevante

O maior valor é:

$2 \rightarrow$ "animal"

Passo 5. Produzir a saída

Selecionamos:

$V_2 = [2,0]$

Passo 6. Decisão final

Suponhamos que temos no vocabulário:

"the" $\rightarrow [1,0]$

"animal" $\rightarrow [0,1]$

"didn't" $\rightarrow [1,1]$

"cross" $\rightarrow [2,0]$

"street" $\rightarrow [1,2]$

Comparando o vetor final [2,0] com os vetores do vocabulário, o mais semelhante é:

"cross"

Projeto 1: Primeiros módulos

Neste primeiro projeto, irão implementar um conjunto de operações fundamentais sobre vetores e memória, que serão utilizadas no Projeto 2 para construir o Mini-Transformer.

O objetivo é consolidar o domínio de:

- operações sobre vetores em Assembly RISC-V;
- manipulação de memória através de ponteiros;
- controlo de fluxo (condições e ciclos).

Cada exercício corresponde a uma operação essencial do modelo de atenção, como comparar vetores ou selecionar resultados. Cada exercício deve ser implementado como um programa autónomo, pelo que a entrega consistirá em vários ficheiros independentes, um por operação.

Outras indicações úteis:

- Devem utilizar a **extensão de multiplicação de RISC-V** (extensão M).
- Todos os inteiros utilizados no projeto são **inteiros com sinal** (*signed int*, em contraste com *unsigned int*).
- Para submissões de excelência: serão valorizadas as implementações que fazem deteção de *overflow*, isto é, cálculos que ultrapassam os limites dos inteiros.

Seleção de elemento (select)

Esta função, implementada na zona de memória com a etiqueta **select**, **seleciona o elemento de um vetor**. Esta função recebe um vetor de inteiros e um índice e devolve o valor do vetor nessa posição.

- **Argumentos**
  - `a1` (int *): Ponteiro para o início do vetor de inteiros
  - `a2` (int): Número de inteiros no vetor.
  - `a3` (int): índice do elemento
- **Valor de Retorno**
  - `a0` (int): Código de estado de retorno
    - Código 0: sucesso
    - Código 50: argumento inválido (tamanho < 1)
    - Código 100: índice fora dos limites
  - `a1` (int): Valor do elemento

Índice do maior elemento (argmax)

Esta função, implementada na zona de memória com a etiqueta **argmax**, recebe um vetor de inteiros e retorna o **índice do maior elemento**. Se existirem múltiplos elementos com o mesmo valor máximo, deve ser devolvido o menor índice entre eles.

Por exemplo, para o vetor de inteiros `[-6, -1, 6, 1]`, o resultado deve ser `2` (índice de `6`). Para o vetor `[6, 1, 6, 1]`, o resultado deve ser `0`, `pois` o valor `6` aparece pela primeira vez nessa posição.

- **Argumentos**
  - `a1` (int *): Ponteiro para o início do vetor de inteiros.
  - `a2` (int): Número de inteiros no vetor.
- **Valor de Retorno**
  - `a0` (int): Código de estado de retorno
    - Código 0: sucesso
    - Código 50: argumento inválido (tamanho < 1)
  - `a1` (int): Índice do maior elemento do vetor (em caso de empate, o menor índice).

Dot Product (dot)

A função `dot`, implementada na zona de memória com etiqueta **dot**, calcula o produto interno entre dois vetores de inteiros.

Dados dois vetores:

```
A = [a1, a2, ..., an]

B = [b1, b2, ..., bn]
```

O resultado é:

```
dot(A, B) = a1*b1 + a2*b2 + ... + an*bn
```

- **Argumentos**
  - `a1` (int *): Ponteiro para vetor A
  - `a2` (int *): Ponteiro para vetor B
  - `a3` (int): Número de elementos de cada vetor
- **Valores de Retorno**
  - `a0` (int): Código de estado de retorno
    - Código 0: sucesso
    - Código 50: argumento inválido (tamanho < 1)
    - Código 200: *overflow* detectado **(pontos extra)**
  - `a1` (int): Resultado do produto interno

Nota final: As funções implementadas neste projeto serão reutilizadas no Projeto 2, nomeadamente:

- `dot`, para cálculo de relevâncias (Q·K),

- `argmax`, para seleção do elemento mais relevante,
- `select`, para obtenção do valor V correspondente.

Em conjunto, estas funções permitem implementar o núcleo do mecanismo de atenção.

Projeto 2: Compor a Rede Neuronal

Neste projeto, irão construir o **Mini-Transformer completo**, combinando as operações implementadas no Projeto 1. Sempre que possível, devem reutilizar as funções já desenvolvidas no Projeto 1.

O objetivo é implementar o pipeline completo do mecanismo de atenção, permitindo ao modelo prever o próximo token (palavra) de uma frase.

Objetivo

Dada uma sequência de tokens representada por uma matriz de embeddings $\boldsymbol{E}$, o modelo deverá:

1. Obter a representação matricial da frase de entrada;
2. Calcular as matrizes $\boldsymbol{Q}$, $\boldsymbol{K}$ e $\boldsymbol{V}$;
3. Calcular as relevâncias entre palavras;
4. Selecionar a palavra mais relevante para a última posição;
5. Produzir o vetor de saída correspondente;
6. Prever o próximo token com base nesse vetor

Pipeline do modelo

O modelo executa os seguintes passos:

Frase → tokens → matriz E → Q, K, V → scores → argmax → ***seleção*** *→ vetor final → previsão*

*(**Nota:** a função de seleção já foi desenvolvida no Projeto 1.)*

Tarefas a implementar

1. Construção da matriz de embeddings da frase

O programa deve receber como entrada uma frase curta, composta por 3 tokens do vocabulário, separados por espaços.

A partir dessa frase, deve construir a matriz de embeddings E, substituindo cada token pelo vetor correspondente presente na tabela de embeddings fornecida.

Para simplificar o projeto, assumam que:

- a frase tem sempre exatamente 3 tokens;
- todos os tokens pertencem ao vocabulário fornecido.

Esta tarefa inclui, portanto:

- ler a frase de entrada;
- identificar os tokens que a compõem;
- procurar cada token no vocabulário;
- copiar os embeddings correspondentes para a matriz E.

São disponibilizados os seguintes ficheiros:

- o ficheiro com o vocabulário (`vocab.txt`);
- o ficheiro com a tabela de embeddings (`embeddings.txt`);
- template de ficheiro com frases de teste (`input.txt`);
- os ficheiros com os pesos do modelo (`W_Q.txt`, `W_K.txt` e `W_V.txt`);
- template de ficheiro de output (`output.txt`).

Isto implica desenvolver funções para:

- ler frases de um ficheiro: `read_file`;
- converter tokens em índices do vocabulário: `convert_token`;
- construir a matriz ***E*** em memória: `build_E`.

```
# read_file
# a0: endereço do nome do ficheiro (char*)
# a1: buffer de destino
# a2: número máximo de bytes a ler
# return (a0): número de bytes lidos

# convert_token
# a0: endereço do token (char*)
# a1: endereço do vocabulário
# a2: número de tokens no vocabulário
# return (a0): índice do token no vocabulário

# build_E
# a0: endereço da matriz E (output)
# a1: endereço da tabela de embeddings
# a2: endereço do vetor de índices dos tokens
# a3: número de tokens (n)
# a4: dimensão dos embeddings (dim)
# return: -
```

2. *Multiplicação de matrizes*

Implementar funções que calculem:

$$\boldsymbol{Q} = \boldsymbol{E} \times \boldsymbol{W_Q}$$
$$\boldsymbol{K} = \boldsymbol{E} \times \boldsymbol{W_K}$$
$$\boldsymbol{V} = \boldsymbol{E} \times \boldsymbol{W_V}$$

As matrizes devem ser armazenadas em memória como arrays lineares.

Isto implica desenvolver uma função de multiplicação matriz × matriz: `mat_mul`.

```
# mat_mul
# a0: endereço da matriz A
# a1: endereço da matriz B
# a2: endereço da matriz C (output)
# a3: número de linhas de A
# a4: número de colunas de A (= linhas de B)
# a5: número de colunas de B
# return: -
```

3. *Cálculo de relevâncias*

Para a última palavra da frase (posição $n - 1$), calcular:

$$score(j) = Q[n-1] \cdot K[j]$$

para todos os valores de ***j*** (todas as posições da frase).

Isto significa desenvolver a função `compute_scores`.

```
# compute_scores
# a0: endereço da matriz Q
# a1: endereço da matriz K
# a2: endereço do vetor de scores (output)
# a3: número de tokens (n)
# a4: dimensão dos embeddings (dim)
# return: -
```

4. *Produção do vetor de saída*

Usando o índice selecionado no passo anterior, escolher o vetor apropriado da matriz ***V***.

Isto significa desenvolver a função `select_vector`.

```
# select_vector
# a0: endereço da matriz V
# a1: endereço do vetor de output
# a2: índice da linha a selecionar
# a3: dimensão dos embeddings (dim)
# return: -
```

5. *Decisão final*

Comparar o vetor final obtido com todos os embeddings do vocabulário (por exemplo, usando produto interno) e escolher a palavra cujo embedding seja mais semelhante.

A palavra selecionada corresponde à previsão do próximo token. Essa palavra deve ser escrita num ficheiro de output.

Isto significa desenvolver a função `decision`.

```
# decision
# a0: endereço do vetor de output
# a1: endereço da tabela de embeddings
# a2: número de tokens no vocabulário
# a3: dimensão dos embeddings (dim)
# return (a0): índice do token previsto
```

Projeto 3: Acelerador

Neste projeto, irão construir um circuito sequencial, mais concretamente um pequeno processador especializado em algumas das operações fundamentais do Mini-Transformer desenvolvido nos projetos anteriores. Este processador será uma versão simplificada de um acelerador de Inteligência Artificial, semelhante aos que são utilizados em hardware moderno para acelerar modelos baseados em atenção. Em particular, este sistema irá suportar operações essenciais como produtos internos entre vetores e seleção de valores, que estão no núcleo do mecanismo de atenção.

Este processador pode ser visto como uma versão muito simplificada de uma unidade de processamento de tensores (Tensor Processing Unit, TPU). Ao contrário da implementação em software realizada em RISC-V nos projetos anteriores, neste caso as operações serão implementadas diretamente em hardware, permitindo uma execução mais eficiente. O objetivo é compreender como operações de alto nível usadas em modelos de Inteligência Artificial podem ser mapeadas para circuitos digitais. Posteriormente, será disponibilizada uma extensão a este enunciado com os detalhes técnicos necessários para a implementação do processador.

Considerações práticas

O projeto deve ser desenvolvido no simulador **Ripes** (1º e 2º projetos) e ferramenta **Logisim** (3º projeto), que serão apresentados nas aulas laboratoriais. O processador simulado em Ripes deve ser um **RISC-V de 32 bits**, de acordo com a arquitetura estudada nas aulas teóricas e laboratoriais. Pode ser usada a extensão M do RISC-V.

No caso dos 1º e 2º projetos, forneceremos os ficheiros de código iniciais (através do Fénix). As soluções devem **obrigatoriamente** ser construídas sobre esses ficheiros, respeitando a estrutura fornecida inicialmente.

Entregas e avaliação

Os projetos são submetidos no fénix, com os seguintes prazos:

- Projeto 1: 23h59 de 30/abril
- Projeto 2: 23h59 de 22/maio
- Projeto 3: 23h59 de 8/junho

O peso de cada projeto na nota final pode ser consultado nos métodos de avaliação no fenix. Nota mínima de 9,0.

Embora o projeto seja desenvolvido em grupos de 3 estudantes, haverá na semana de discussões um teste prático individual e discussão cujo objetivo é aferir se cada membro do grupo contribuiu e domina as soluções que foram submetidas pelo grupo. O teste individual inclui modificações ao projeto submetido pelo grupo e possivelmente o desenvolvimento de novas funcionalidades.

Usaremos software de deteção de cópias (abarcando uso de AI/ChatGPT). Casos de plágio seguem os procedimentos do regulamento de avaliação do IST.